Terraform on AWS: EC2, S3, Key Pair & Security Group

A hands-on lab that uses **Terraform** to provision and destroy AWS infrastructure from a Windows machine (PowerShell), using a dedicated IAM user.

Item	Value
Date	18 October 2026
Tool	Terraform (AWS provider hashicorp/aws v6.69.1, ~> 6.0)
Cloud	AWS, region us-east-1 (N. Virginia)
Shell	Windows PowerShell
Working directory	D:\github\terraform-modules\terraform

Objectives

1. Create an IAM user with programmatic access for Terraform.
2. Write Terraform configuration for an EC2 instance, key pair, security group and S3 bucket.
3. Run the Terraform workflow: init, validate, plan, apply.
4. Verify the resources in the AWS Console.
5. Clean everything up with terraform destroy.

1. IAM user and permissions

An IAM user named **terraform-lab-user** was created with console access disabled and one active access key (used only by Terraform). Three AWS managed policies are attached directly: AmazonEC2FullAccess, AmazonEC2ReadOnlyAccess and AmazonS3FullAccess.

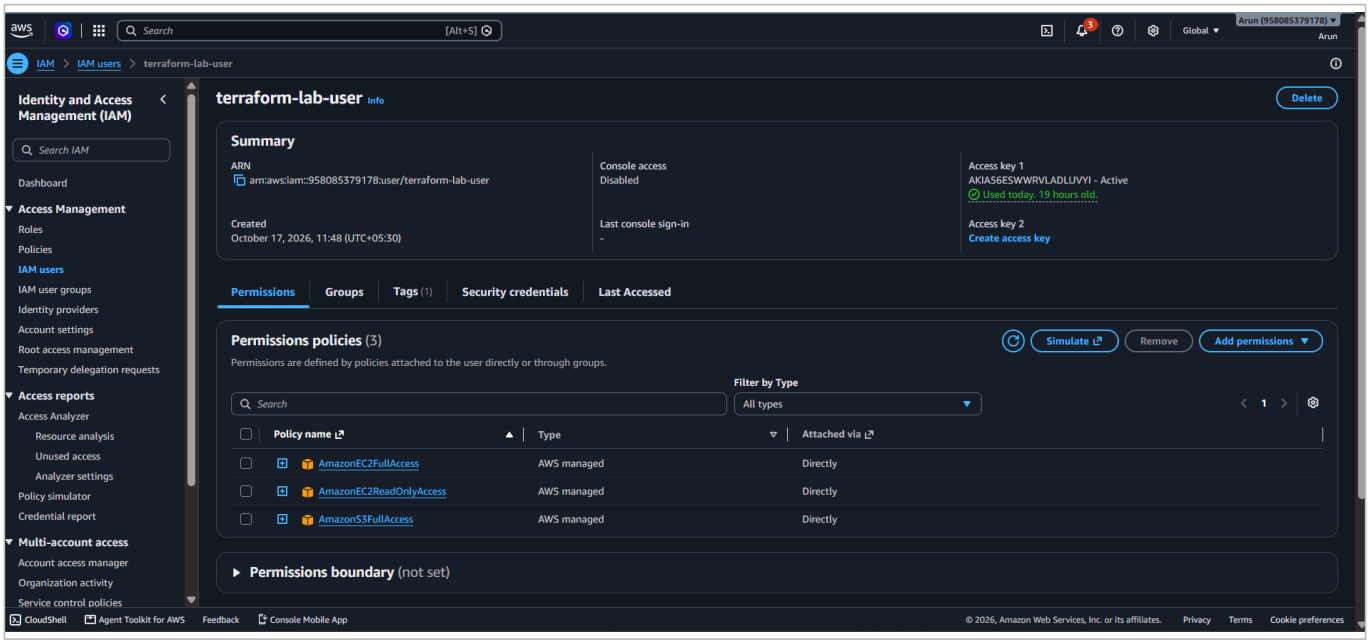


Figure 1: IAM user with permissions

Note: the first terraform plan failed with UnauthorizedOperation for ec2:DescribeImages (section 3). Attaching AmazonEC2ReadOnlyAccess fixed it.

2. Terraform project files

File	Purpose
provider.tf	AWS provider configuration (~> 6.0, region us-east-1)
instance.tf	aws_instance (t3.micro) and Ubuntu AMI data source
keypair.tf	aws_key_pair (deployer-key)
security-group.tf	Security group plus ingress (TLS 443, IPv4/IPv6) and egress rules
s3.tf	S3 bucket with tags
.terraform.lock.hcl	Provider version lock file
terraform.tfstate / .backup	Local state (generated by Terraform)

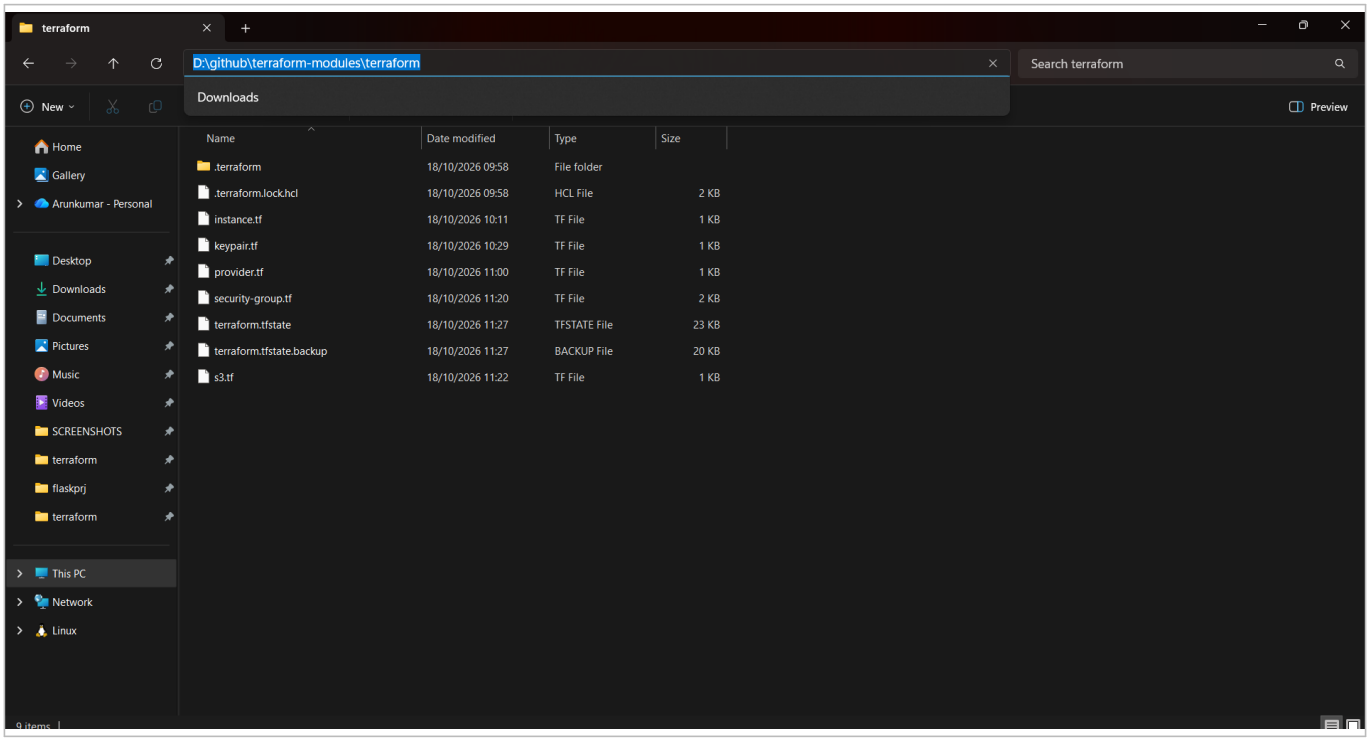


Figure 2: Terraform project files

3. Provisioning the EC2 instance

3.1 Initialize, validate, and the first plan error

The files were first saved as .tf.txt, so they were renamed with Rename-Item. Then terraform init installed the AWS provider and terraform validate passed. terraform plan then failed with a **403 UnauthorizedOperation**: the IAM user was not allowed to run ec2:DescribeImages. This was resolved by adding the AmazonEC2ReadOnlyAccess policy.

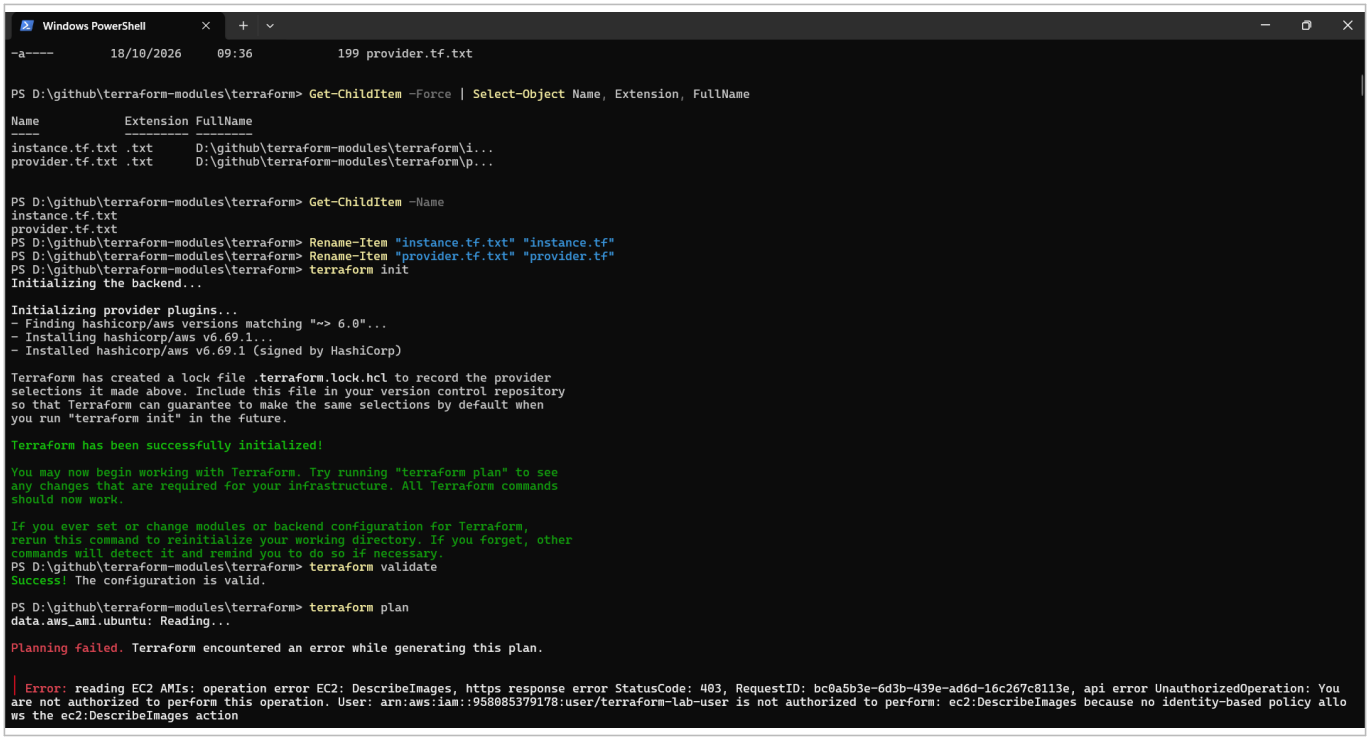


Figure 3: terraform init, validate and the plan error

3.2 terraform apply: execution plan

After the permission fix, the data source read the Ubuntu AMI (ami-09db1e291cef3f1ef) and Terraform planned one aws_instance.example (t3.micro, us-east-1).

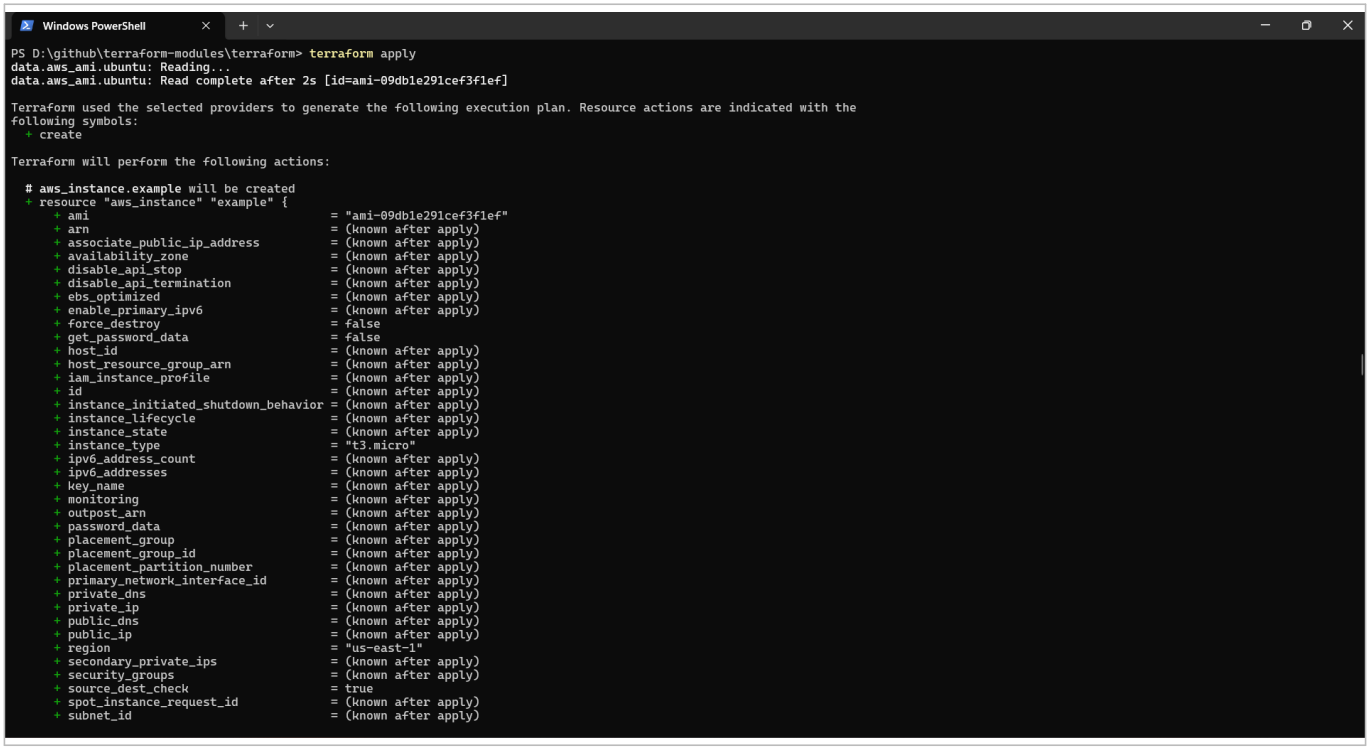


Figure 4: terraform apply, execution plan

3.3 Apply complete

After confirming with yes, the instance was created in 17 seconds. **Result: Apply complete! Resources: 1 added, 0 changed, 0 destroyed.**

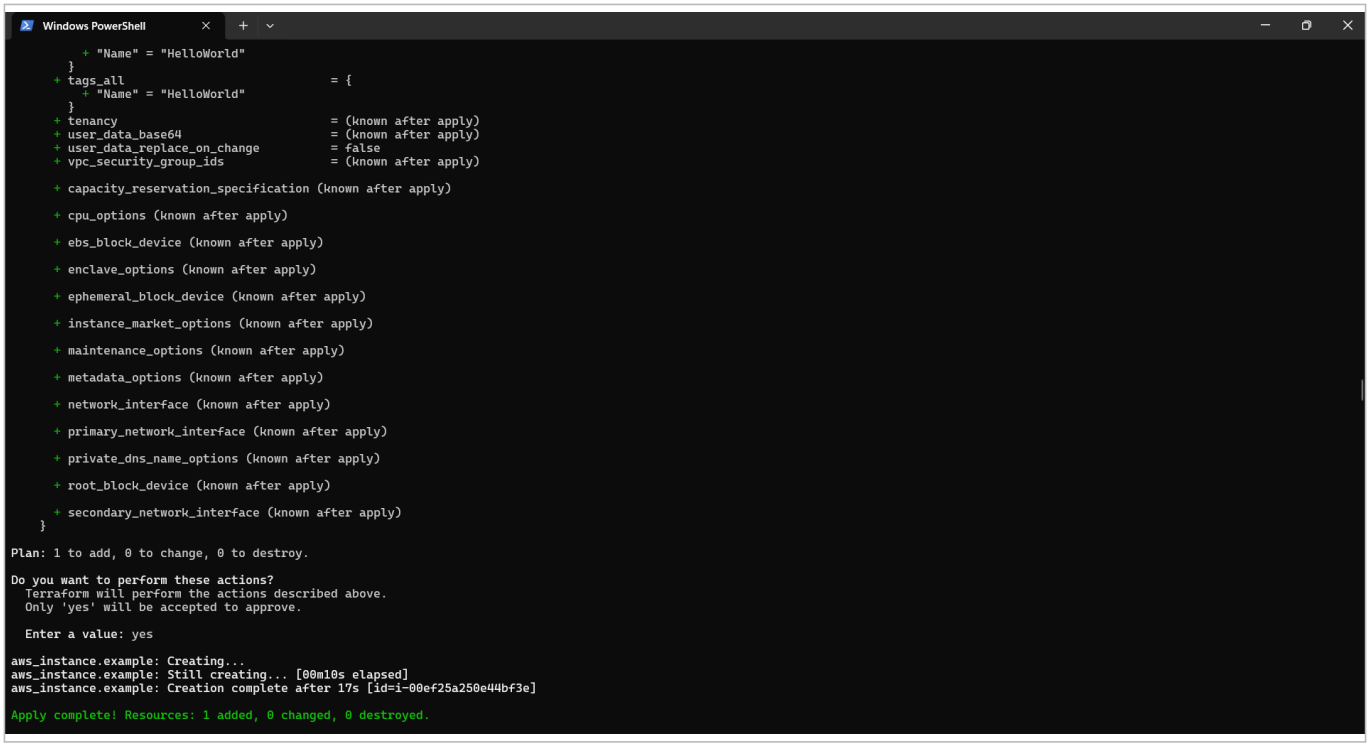


Figure 5: terraform apply complete

3.4 Verification in the AWS Console

The HelloWorld instance is in the **Running** state with **3/3 status checks passed**, type t3.micro, in us-east-1c.

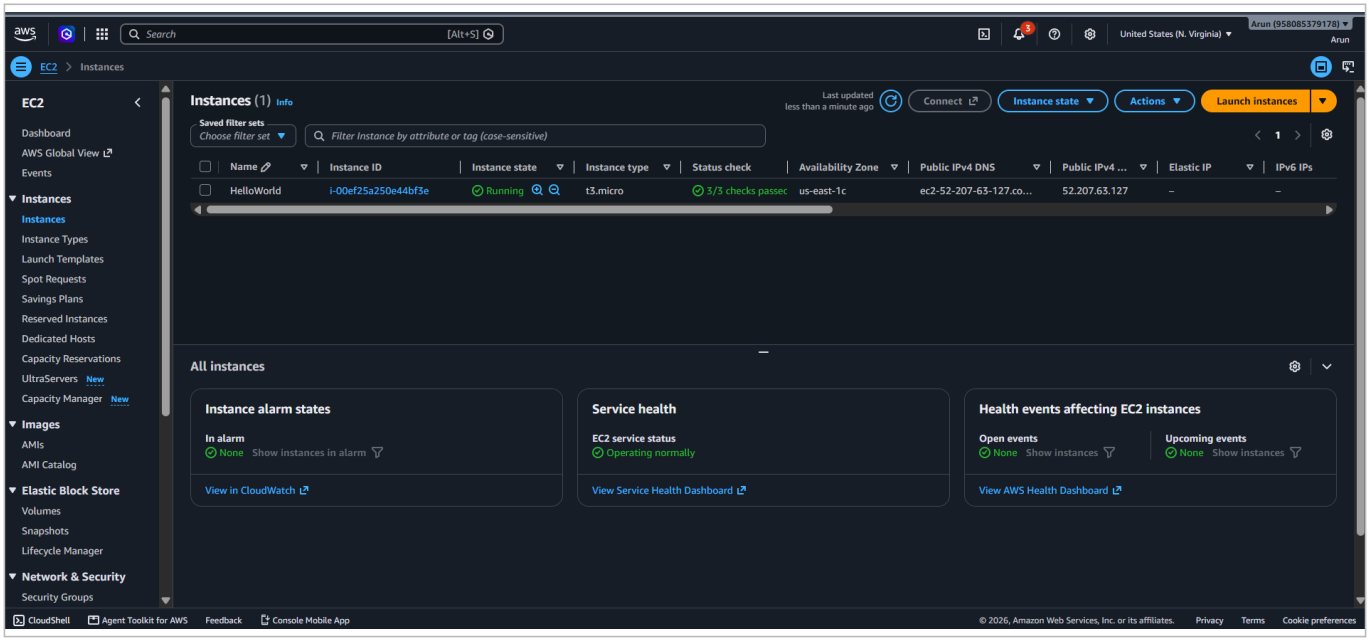


Figure 6: EC2 instance running in the AWS Console

4. Adding the security group, S3 bucket and key pair

4.1 Re-initialize and fix validation errors

After adding the new files, terraform init -reconfigure and terraform fmt were run. terraform validate reported **Reference to undeclared resource** in s3.tf because the data sources aws_caller_identity.current and aws_region.current were not declared. After declaring them, validation passed.

```
PS D:\github\terraform-modules\terraform> terraform init --reconfigure
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.69.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS D:\github\terraform-modules\terraform> terraform fmt
provider.tf
PS D:\github\terraform-modules\terraform> terraform validate

  Error: Reference to undeclared resource

    on s3.tf line 2, in resource "aws_s3_bucket" "example":
     2:   bucket          = format("my-tf-test-bucket-%s-%s-an", data.aws_caller_identity.current.account_id, data.aws_region.current.region)

  A data resource "aws_caller_identity" "current" has not been declared in the root
  module.


  Error: Reference to undeclared resource

    on s3.tf line 2, in resource "aws_s3_bucket" "example":
     2:   bucket          = format("my-tf-test-bucket-%s-%s-an", data.aws_caller_identity.current.account_id, data.aws_region.current.region)

  A data resource "aws_region" "current" has not been declared in the root module.

PS D:\github\terraform-modules\terraform> terraform validate
Success! The configuration is valid.

PS D:\github\terraform-modules\terraform> terraform init --reconfigure
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.69.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
```

Figure 7: init -reconfigure, fmt and validate

4.2 Apply: S3 bucket, security group and key pair

The S3 bucket my-tf-test-bucketqwerty was created (tags: Environment = Dev, Name = My bucket). **Result: Apply complete! Resources: 1 added, 0 changed, 0 destroyed.** The security group (with its ingress/egress rules) and the key pair deployer-key were created through the same workflow.

```
Windows PowerShell

      + hosted_zone_id               = (known after apply)
      + id                           = (known after apply)
      + object_lock_enabled          = (known after apply)
      + policy                       = (known after apply)
      + region                       = "us-east-1"
      + request_payer                = (known after apply)
      + tags                         = {
          + "Environment" = "Dev"
          + "Name"        = "My bucket"
        }
      + tags_all                     = {
          + "Environment" = "Dev"
          + "Name"        = "My bucket"
        }
      + website_domain               = (known after apply)
      + website_endpoint             = (known after apply)

      + cors_rule (known after apply)

      + grant (known after apply)

      + lifecycle_rule (known after apply)

      + logging (known after apply)

      + object_lock_configuration (known after apply)

      + replication_configuration (known after apply)

      + server_side_encryption_configuration (known after apply)

      + versioning (known after apply)

      + website (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_s3_bucket.main: Creating...
aws_s3_bucket.main: Creation complete after 7s [id=my-tf-test-bucketqwerty]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS D:\github\terraform-modules\terraform>
```

Figure 8: S3 bucket, security group and key pair created

5. Cleaning up with terraform destroy

Terraform refreshed the state and planned to remove all **9** managed resources: the EC2 instance, key pair deployer-key, S3 bucket my-tf-test-bucketqwerty, security group allow_tls, 4 security group rules (2 ingress, 2 egress) and VPC main.

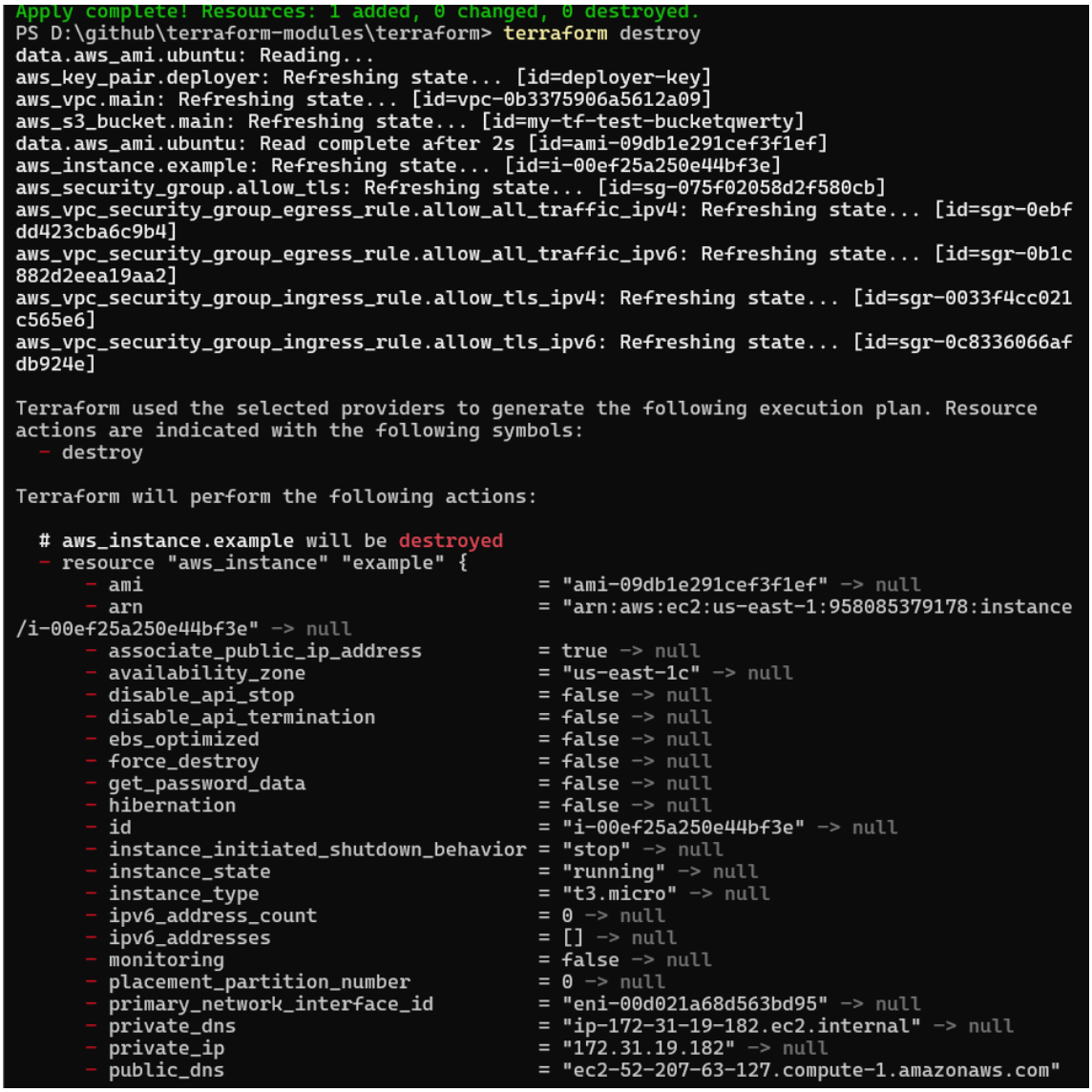


Figure 9: terraform destroy, plan

After typing yes, all resources were removed. **Result: Destroy complete! Resources: 9 destroyed.**

```
      - arn                    = "arn:aws:ec2:us-east-1:958085379178:security-group-rule/sg
r-0c8336066afdb924e" -> null
      - cidr_ipv6              = "::/0" -> null
      - from_port              = 443 -> null
      - id                     = "sgr-0c8336066afdb924e" -> null
      - ip_protocol            = "tcp" -> null
      - region                 = "us-east-1" -> null
      - security_group_id      = "sg-075f02058d2f580cb" -> null
      - security_group_rule_id = "sgr-0c8336066afdb924e" -> null
      - tags_all               = {} -> null
      - to_port                = 443 -> null
    }

Plan: 0 to add, 0 to change, 9 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_vpc_security_group_ingress_rule.allow_tls_ipv4: Destroying... [id=sgr-0033f4cc021c565e6
]
aws_vpc_security_group_egress_rule.allow_all_traffic_ipv4: Destroying... [id=sgr-0ebfdd423c
ba6c9b4]
aws_vpc_security_group_egress_rule.allow_all_traffic_ipv6: Destroying... [id=sgr-0b1c882d2e
ea19aa2]
aws_vpc_security_group_ingress_rule.allow_tls_ipv6: Destroying... [id=sgr-0c8336066afdb924e
]
aws_key_pair.deployer: Destroying... [id=deployer-key]
aws_s3_bucket.main: Destroying... [id=my-tf-test-bucketqwerty]
aws_instance.example: Destroying... [id=i-00ef25a250e44bf3e]
aws_key_pair.deployer: Destruction complete after 1s
aws_s3_bucket.main: Destruction complete after 1s
aws_vpc_security_group_egress_rule.allow_all_traffic_ipv4: Destruction complete after 1s
aws_vpc_security_group_ingress_rule.allow_tls_ipv6: Destruction complete after 1s
aws_vpc_security_group_ingress_rule.allow_tls_ipv4: Destruction complete after 1s
aws_vpc_security_group_egress_rule.allow_all_traffic_ipv6: Destruction complete after 1s
aws_security_group.allow_tls: Destroying... [id=sg-075f02058d2f580cb]
aws_security_group.allow_tls: Destruction complete after 2s
aws_vpc.main: Destroying... [id=vpc-0b3375906a5612a09]
aws_vpc.main: Destruction complete after 1s
aws_instance.example: Still destroying... [id=i-00ef25a250e44bf3e, 00m10s elapsed]
aws_instance.example: Still destroying... [id=i-00ef25a250e44bf3e, 00m20s elapsed]
aws_instance.example: Still destroying... [id=i-00ef25a250e44bf3e, 00m30s elapsed]
aws_instance.example: Destruction complete after 32s

Destroy complete! Resources: 9 destroyed.
PS D:\github\terraform-modules\terraform>
```

Figure 10: terraform destroy complete

Summary

Stage	Command	Outcome
Setup	terraform init	Provider v6.69.1 installed
Check	terraform validate	Configuration valid
Plan	terraform plan	Failed at first (403), fixed with EC2 ReadOnly policy
Create	terraform apply	EC2, then S3, security group and key pair created
Verify	AWS Console	Instance running, 3/3 checks passed
Clean up	terraform destroy	9 resources destroyed

Issues faced and fixes

Issue	Cause	Fix
Files not recognized by Terraform	Saved as .tf.txt	Renamed with Rename-Item
UnauthorizedOperation on ec2:DescribeImages	IAM user lacked describe permissions	Attached AmazonEC2ReadOnlyAccess

Issue	Cause	Fix
Reference to undeclared resource	Missing aws_caller_identity and aws_region data sources	Declared both in the configuration

Key learnings

• The Terraform workflow is init, validate, plan, apply, destroy.

• Read the exact action in IAM errors (for example ec2:DescribeImages) and grant only what is needed.

• terraform destroy removes everything tracked in state, so it avoids leftover cost.

• Fix file extensions and formatting early (terraform fmt).